

DIGITAL THERMOMETER USING 8051 MICROCONTROLLER

Team members:

Kurian sony(31)

KTU ID:MDL23EC073

Peter Ajith Cherian (47)

KTU ID:MDL23EC101

CONTENTS

- PAGE 3: INTRODUCTION
- PAGE 4: LITERATURE REVIEW
- PAGE 5: DESIGN
- PAGE 13: ALGORITHM
- PAGE 14: HARDWARE REQUIREMENTS
- PAGE 15: SOFTWARE REQUIREMENTS
- PAGE 16: DESIGN METHODOLOGY
- PAGE 18: INSTALLATION OF SOFTWARE TOOLS
- PAGE 20: RESULTS
- PAGE 22: CONCLUSION
- PAGE 23: FUTURE SCOPE
- PAGE 24: REFERENCES

INTRODUCTION

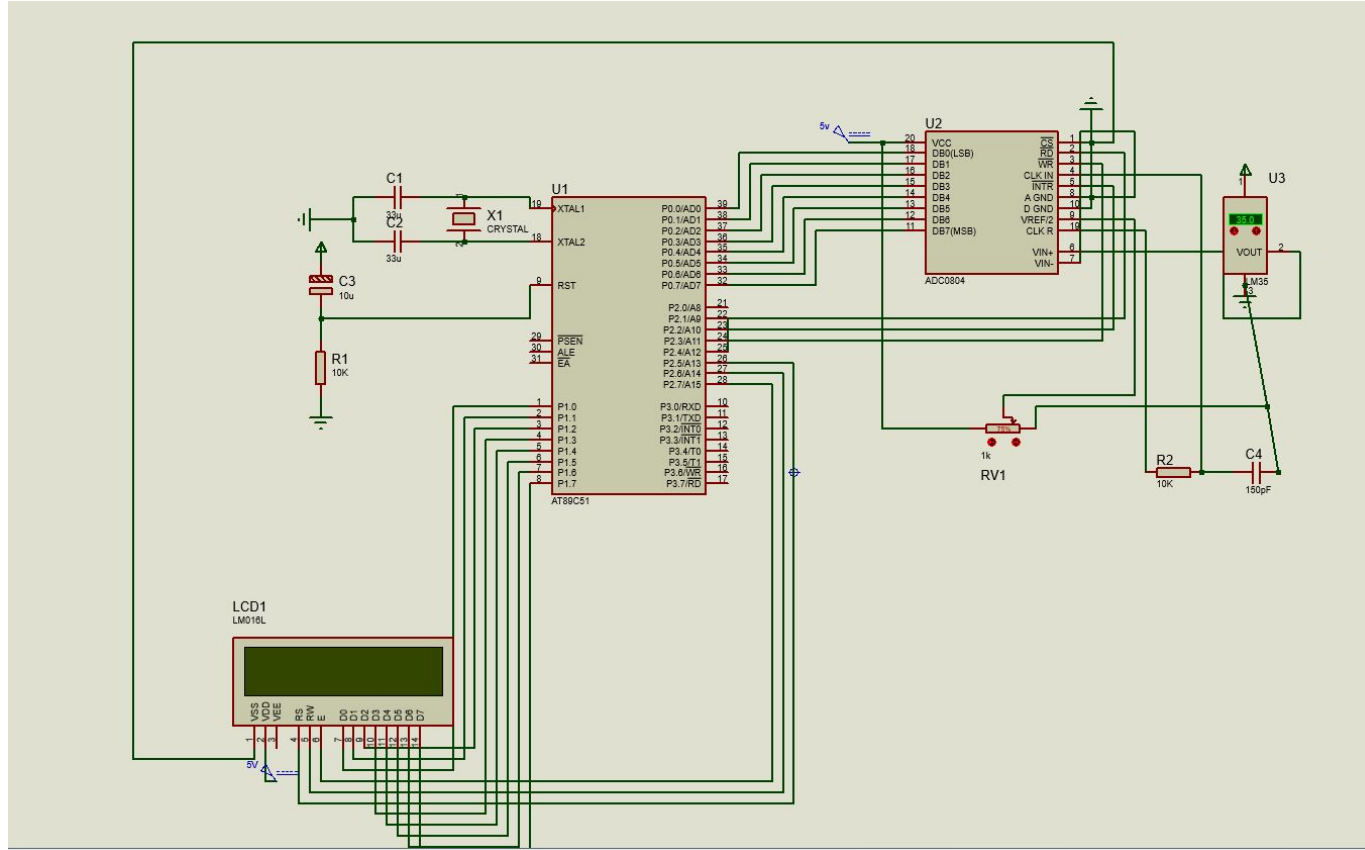
This project aims to design and implement a digital thermometer using the 8051 microcontroller. The 8051 microcontroller, a widely used microcontroller in embedded systems, is chosen for its simplicity, cost-effectiveness, and ease of interfacing with sensors and display units. The thermometer will use a temperature sensor to detect ambient temperature and convert the analog readings into a digital signal. The microcontroller will process this signal, calibrate the data, and display the temperature on an LCD screen in real time.

LITERATURE REVIEW

The 8051 microcontroller has been extensively utilized in developing temperature monitoring systems across various applications. For instance, a project detailed by EngineersGarage demonstrates measuring room temperature using an LM35 sensor interfaced with an 89c51 microcontroller. The system employs an external ADC0804 to convert the analog signal from the LM35 into a digital format, which is then processed by the microcontroller and displayed on a 16×2 LCD. This setup effectively monitors room temperature in both Celsius and Fahrenheit scales.

In another application, Elektor Magazine presented a six-channel temperature monitor and logger capable of tracking temperatures ranging from -240°C to +850°C. This system utilizes Pt100 RTDs for precise measurements and is based on the 8051 microcontroller, making it suitable for industrial applications requiring multi-point temperature monitoring.

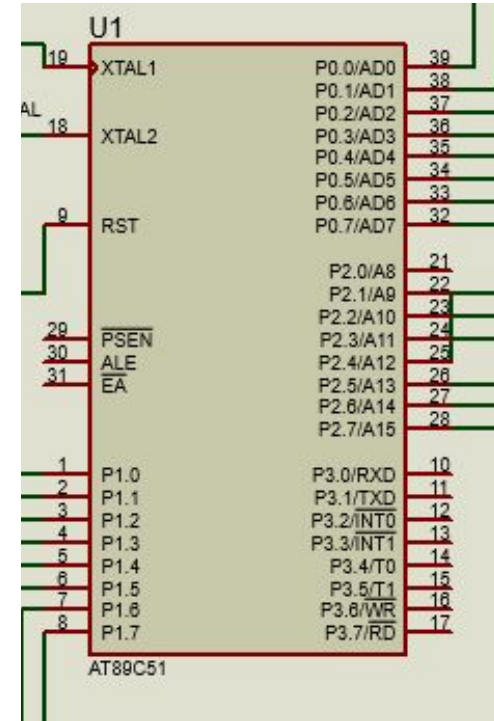
DESIGN



DESIGN

AT89C51 Microcontroller (U1)

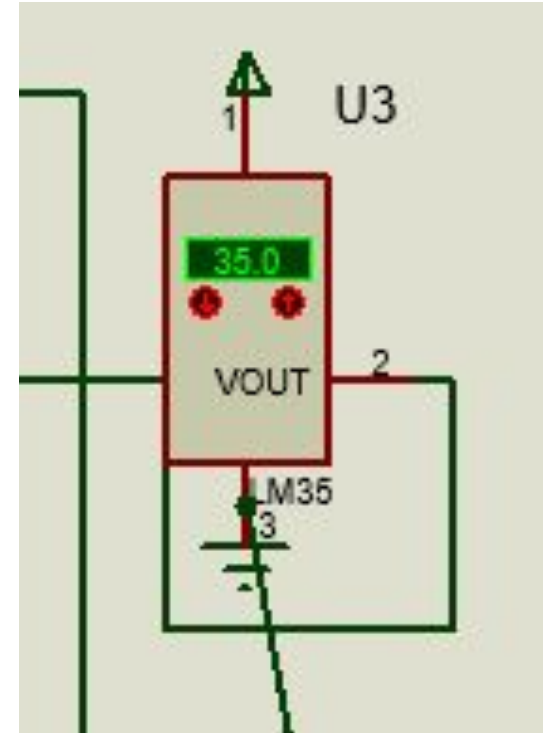
- **Purpose:**
This is the core processing unit that receives digital data from the ADC and processes it to calculate the temperature. It also controls the LCD display.
- **Key Pins:**
 - **XTAL1 & XTAL2:** Connected to a crystal oscillator (X1) and capacitors (C1, C2) to provide the clock signal for the microcontroller's operation.
 - **RESET Pin (Pin 9):** Connected to the capacitor (C3) and resistor (R1) to ensure proper initialization of the microcontroller during power-up.
 - **Port 0 (Pins 32-39):** Used for data communication between the microcontroller and ADC0804.
 - **Port 2 (Pins 21-28):** Connected to the control lines of ADC0804.
 - **Port 1 (Pins 1-8):** Interfaced with the 16x2 LCD to send data and control sign



DESIGN

LM35 Temperature Sensor (U3)

- **Purpose:**
This sensor measures the ambient temperature and provides an analog output voltage proportional to the temperature (10 mV per °C).
- **Connections:**
 - **VOUT Pin:** Provides the analog output, which is connected to the ADC0804 for analog-to-digital conversion.
 - **VCC and GND Pins:** Provide power to the sensor.



DESIGN

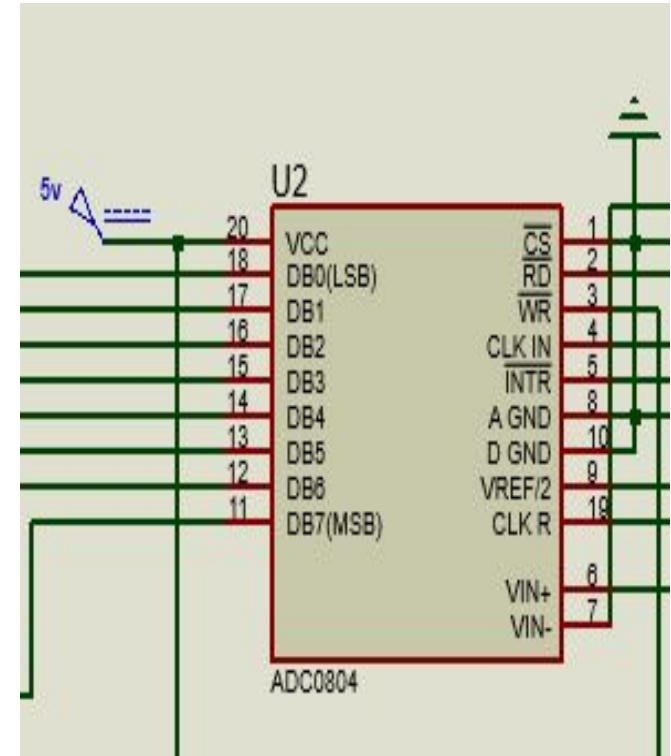
ADC0804 (U2) – Analog-to-Digital Converter

- **Purpose:**

The ADC0804 converts the analog signal from the LM35 into an 8-bit digital value that the 8051 microcontroller can process.

- **Key Pins:**

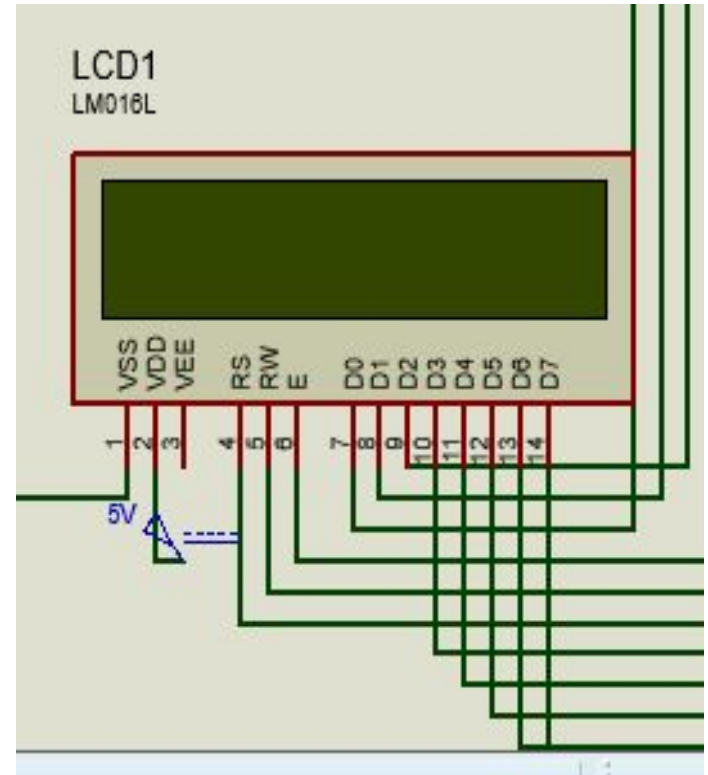
- **VIN(+)/VIN(-):** Connected to the output of LM35 to receive the analog temperature signal.
- **DB0-DB7 (Pins 11-18):** 8-bit digital output pins that send the converted digital data to the microcontroller's Port 0.
- **CS, RD, WR, INTR (Control Pins):** Used for interfacing and controlling ADC operations.
- **VREF/2 Pin:** Used to set the reference voltage for ADC conversion. A resistor divider is used to adjust this value (R2 and RV1).
- **CLK IN:** Connected to an external clock circuit (R2 and C4) to provide the necessary clock signal for ADC operation.



DESIGN

16x2 LCD Display (LCD1)

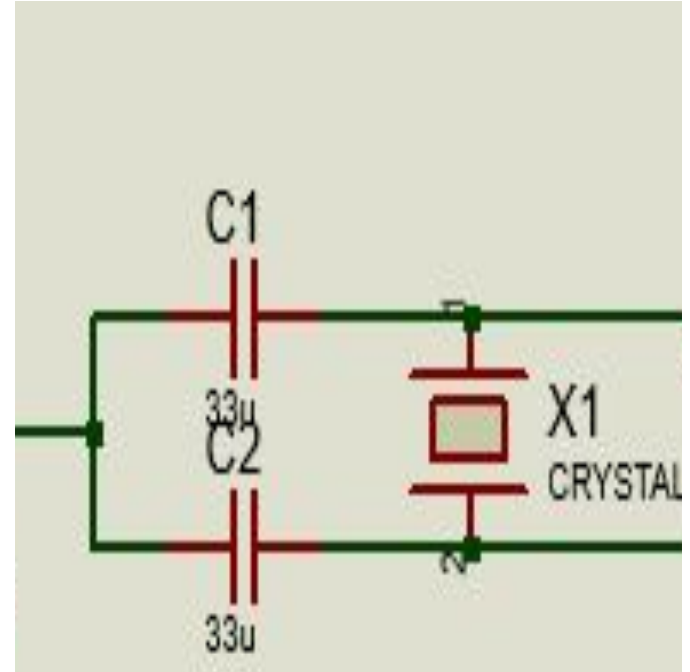
- **Purpose:**
Displays the processed temperature value in real-time.
- **Key Pins:**
 - **RS, RW, EN:** Control pins used for data/command selection, read/write mode, and enable signal, respectively.
 - **DB0-DB7:** Data pins connected to Port 1 of the microcontroller for sending data to be displayed.
 - **VSS, VCC, and VEE:** Provide power to the LCD and adjust contrast using a potentiometer (RV1).



DESIGN

. Crystal Oscillator (X1) and Capacitors (C1, C2)

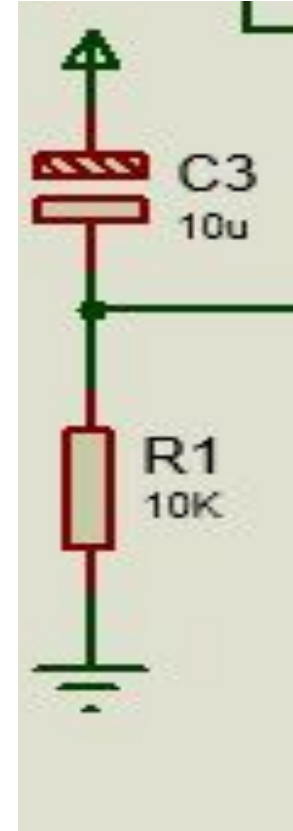
- **Purpose:**
Provides the necessary clock frequency (commonly 11.0592 MHz) for the operation of the AT89C51 microcontroller.



DESIGN

Reset Circuit (R1, C3)

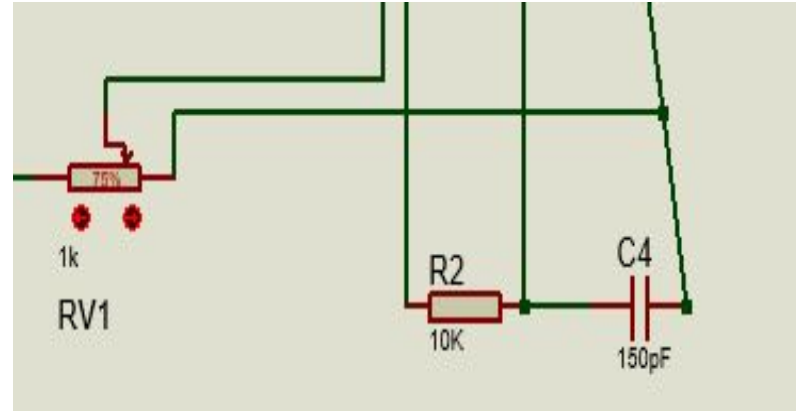
1. **Purpose:** Ensures proper initialization of the ATmega328P during power-up or after a manual reset, preventing undefined behavior.
2. **Components:** Includes a push-button switch, pull-up resistor (10k Ω), and optional capacitor for signal stability and debounce.
3. **Operation:** Pressing the reset button pulls the reset pin low to trigger a reset, while the pull-up resistor restores it to high for normal operation.



DESIGN

Resistors and Capacitors (R2, RV1, C4)

- **Purpose:**
 - **R2 and C4:** Form a clock circuit for the ADC0804.
 - **RV1 (Potentiometer):** Adjusts the reference voltage of the ADC and LCD contrast.



ALGORITHM

1. Initialize ports and define control pins for ADC and LCD.
2. Define functions for delay, sending LCD commands, displaying data, reading ADC values, and displaying strings on the LCD.
3. Initialize the LCD with appropriate commands (0x38, 0x0C, 0x01, 0x80).
4. Enter an infinite loop (while(1)):
 - Set P0 as input for ADC.
 - Display the string "temp:" on the LCD.
 - Read the ADC value by calling the adc() function.
 - Convert the ADC value into separate digits (thousands, hundreds, tens, and units).
 - Display each digit on the LCD after converting to ASCII by adding 0x30.
 - Display the degree symbol (0xDF) and the letter "c" for degrees Celsius.
5. Repeat the loop to continuously read and display updated temperature readings.

HARDWARE REQUIREMENTS

1. **Microcontroller:** 8051 (AT89C51/AT89S51)
2. **Temperature Sensor:** LM35
3. **Analog-to-Digital Converter (ADC):** ADC0804
4. **LCD Display:** 16x2 LCD
5. **Pull-Up Resistor:** 10k Ω
6. **Decoupling Capacitors:** 10 μ F, 100nF
7. **Variable Resistor (for LCD Contrast):** 10k Ω
8. **Push-Button Switch:** SPST Switch
9. **Power Supply:** 5V DC, 7805 Voltage Regulator
10. **Crystal Oscillator:** 11.0592 MHz

SOFTWARE REQUIREMENTS

Keil μ Vision IDE – For writing, compiling, and debugging the 8051 code.

Proteus Design Suite – For circuit simulation and testing.

8051 Compiler (included in Keil) – To compile the code into machine language.

Flash Magic / ISP Programmer Tool – For burning the hex file into the 8051 microcontroller.

Assembler (if needed) – Converts assembly language code into machine code (e.g., A51 assembler).

DESIGN METHODOLOGY

- **Problem Definition and Requirements Analysis:**
 - Identify the need for a digital thermometer capable of measuring and displaying real-time temperature.
 - Define the hardware and software requirements based on functionality (e.g., LM35 sensor, ADC0804, 16x2 LCD, 8051 microcontroller).
- **Block Diagram Design:**
 - Create a block diagram representing the overall system, including the temperature sensor (LM35), ADC0804, microcontroller (8051), LCD display, and power supply.
- **Circuit Design:**
 - Design the circuit by connecting the LM35 sensor to the input of ADC0804.
 - Connect the ADC data lines to the 8051 microcontroller's input port.
 - Interface the 16x2 LCD with the microcontroller for displaying the temperature.
 - Include the reset circuit, oscillator circuit (crystal oscillator with capacitors), and power supply connections.

- **Code Development:**
 - Write the embedded C code for 8051 to read ADC data, process the temperature value, and display it on the LCD.
 - Implement functions for ADC reading, LCD initialization, and temperature data display.
- **Simulation and Testing:**
 - Simulate the designed circuit using software like Proteus to verify the working of the ADC, microcontroller, and LCD display.
 - Debug the code to fix any issues detected during simulation.
- **Hardware Implementation:**
 - Assemble the hardware on a breadboard or PCB by connecting the components as per the designed circuit diagram.
 - Upload the compiled hex code to the 8051 microcontroller using a programming tool (e.g., Flash Magic).
- **Testing and Validation:**
 - Power up the system and test its functionality by observing the temperature displayed on the LCD.
 - Validate the accuracy of the temperature readings by comparing them with a standard thermometer.
- **Documentation and Finalization:**
 - Document the project design, implementation steps, code, and test results.
 - Implement any final adjustments or optimizations for better performance and accuracy.

INSTALLATION DETAILS OF SOFTWARE TOOLS

Keil μ Vision IDE Installation

- Download the Installer:
 - Visit the Keil Product Download Hub and select the appropriate version of the Keil MDK for your system.
- Run the Installer:
 - Locate the downloaded file and double-click to initiate the installation.
- Follow On-Screen Prompts:
 - Accept the license agreement.
 - Choose the installation directory.
 - Complete the installation by following the prompts.

Proteus Design Suite Installation

- Download the Installer:
 - Navigate to the Labcenter Electronics Downloads page and select the appropriate version of Proteus for your needs.
- Run the Installer:
 - Double-click the downloaded executable to start the setup.
- Complete the Setup:
 - Follow the on-screen instructions to install Proteus on your system.

RESULTS

Experiment Overview

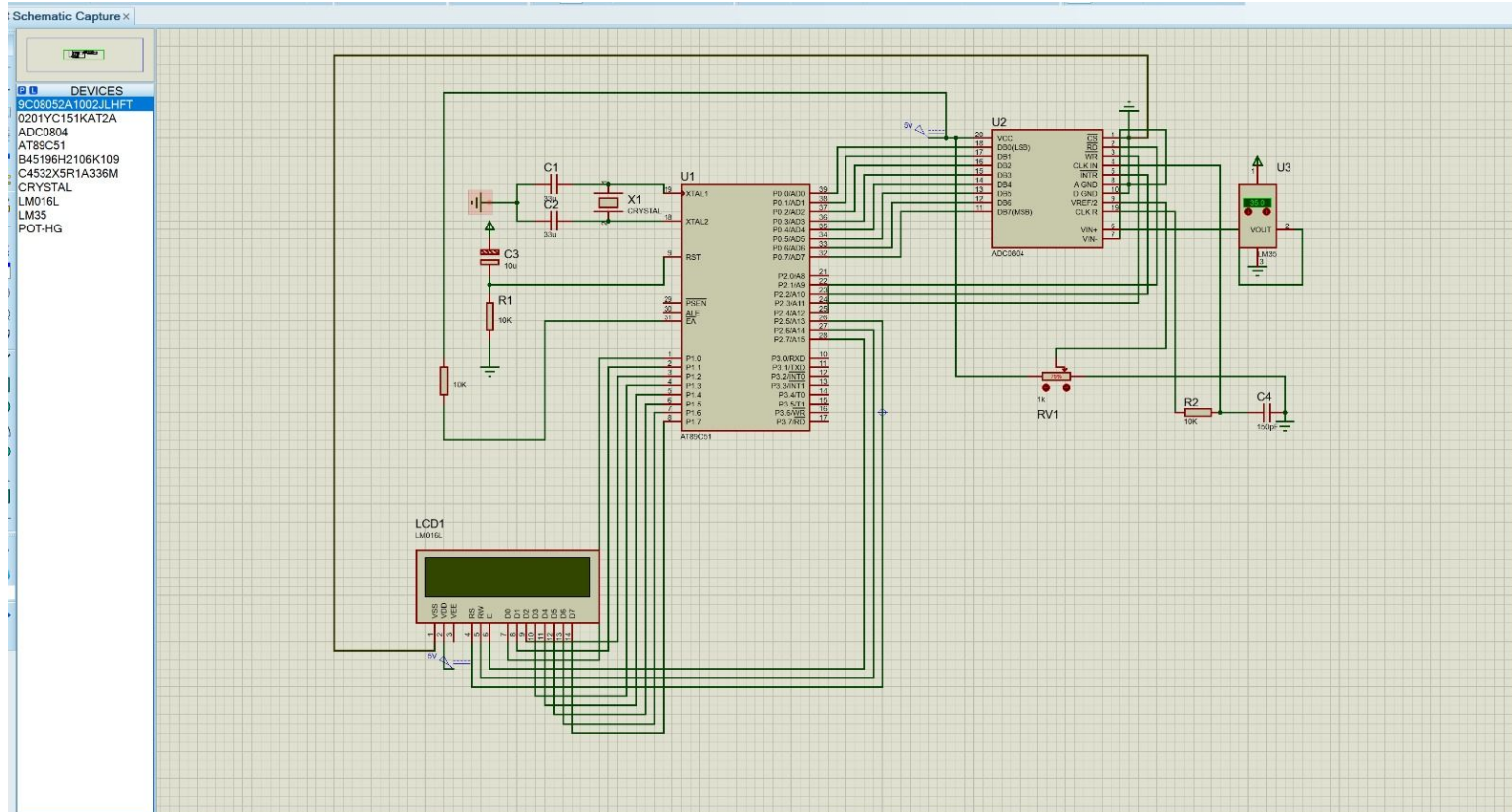
The digital thermometer project was successfully implemented using an 8051 microcontroller, ADC0804 for analog-to-digital conversion, and an LM35 temperature sensor to measure temperature. The results were displayed on an LCD (LM016L), showing the real-time temperature reading.

Simulation Results

1. **Temperature Measurement:** The **LM35 sensor** accurately converted ambient temperature into a proportional voltage, which was then processed by the **ADC0804**.
2. **Analog-to-Digital Conversion:** The **ADC0804** converted the analog signal from the LM35 sensor into a digital output, which was read by the **8051 microcontroller**.
3. **LCD Display Output:** The processed digital temperature value was displayed on the **16x2 LCD screen** in degrees Celsius.
4. **Real-time Updates:** The system updated the temperature reading continuously in response to changes in the environment.

RESULTS

Screenshots of Simulation Results



CONCLUSION

The digital thermometer using the 8051 microcontroller is an efficient and cost-effective system for real-time temperature measurement and display. By interfacing the LM35 temperature sensor, ADC0804, and a 16x2 LCD, the microcontroller processes the analog input, converts it to a digital value, and displays the corresponding temperature in degrees Celsius. This project demonstrates essential concepts of embedded systems, including ADC interfacing, peripheral control, and LCD communication. The design ensures stable operation through the use of reset and oscillator circuits. With potential applications in weather monitoring, home automation, medical equipment, and industrial control, this project highlights the practical use of microcontroller-based temperature monitoring. It provides a solid foundation for further improvements, such as adding wireless connectivity, data logging, or IoT integration.

FUTURE SCOPE

- **Integration with IoT:** Enhancing the system with wireless communication modules can enable remote monitoring and control, facilitating real-time data access and management. [1]
- **Implementation of Advanced Control Algorithms:** Incorporating sophisticated control strategies, such as Proportional-Integral-Derivative (PID) controllers, can improve system response and stability in temperature regulation applications.[2]
- **Adoption of Modern Microcontrollers:** Transitioning to contemporary microcontrollers with enhanced capabilities can provide more efficient processing and additional features, thereby expanding the system's functionality. [3]
- **Multi-Parameter Monitoring:** Extending the system to monitor additional environmental parameters, such as humidity and light intensity, can broaden its applicability in various domains, including agricultural and industrial settings. [4]
- **Development of User-Friendly Interfaces:** Creating intuitive interfaces, such as mobile applications or web dashboards, can enhance user interaction and accessibility, allowing for seamless monitoring and control.[5]

REFERENCES

- [1] N. Elqeblawy, A. Mohammed, and H. A. Hefny, "A Proposed Fuzzy Logic Approach for Conserving the Energy of Data Transmission in the Temperature Monitoring Systems of Internet of Things," *arXiv preprint arXiv:2204.05739*, Apr. 2022.
- [2] K. Rao, W. Song, S. Yalamanchili, and Y. Wardi, "Temperature Regulation in Multicore Processors Using Adjustable-Gain Integral Controllers," *arXiv preprint arXiv:1507.06357*, Jul. 2015.
- [3] Y. Li, Y. Wen, K. Guan, and D. Tao, "Transforming Cooling Optimization for Green Data Center via Deep Reinforcement Learning," *arXiv preprint arXiv:1709.05077*, Sep. 2017.
- [4] T. L. Narayana, C. Venkatesh, A. Kiran, C. B. J, A. Kumar, S. B. Khan, A. Almusharraf, and M. T. Quasim, "Advances in Real-Time Smart Monitoring of Environmental Parameters Using IoT and Sensors," *Heliyon*, vol. 10, no. 4, pp. e28195, Apr. 2024.
- [5] D. Witczak and S. Szymoniak, "Review of Monitoring and Control Systems Based on Internet of Things," *Applied Sciences*, vol. 14, no. 19, pp. 8943, Oct. 2024